

Universidad
Rey Juan Carlos

ESCUELA TÉCNICA SUPERIOR
DE INGENIERÍA INFORMÁTICA

GRADO EN INTELIGENCIA ARTIFICIAL

Práctica 2: Reconocimiento de Texto

VISIÓN ARTIFICIAL

CURSO 2024-2025

**Redactado por: Carmen Maeso Soler
Daniel del Nuevo Montero**

Índice

1. Introducción y estructura del proyecto	2
2. Ejercicio 1: Entrenamiento y validación del clasificador de caracteres	3
2.1. Descripción del conjunto de datos	3
2.2. Carga y preprocesamiento de imágenes	3
2.3. Entrenamiento del clasificador	4
2.4. Evaluación del modelo	5
2.4.1. Métricas de evaluación	5
2.4.2. Curva ROC del modelo SVM	6
2.4.3. Curva Precision-Recall del modelo SVM	6
3. Ejercicio 2: Pruebas con alternativas	7
3.1. Nuevos modelos	7
3.1.1. Métricas comparativas de modelos	7
3.1.2. Curva ROC de los modelos	7
3.1.3. Curva Precision-Recall de los modelos	8
3.2. Comparativa de modelos: SVM, KNN y Random Forest	9
3.3. Análisis de métricas de rendimiento	9
3.3.1. Análisis de las curvas ROC y Precision-Recall	10
3.3.2. Tiempo de entrenamiento	10
3.3.3. Conclusión	10

1. Introducción y estructura del proyecto

El objetivo de la práctica es un sistema OCR modular para reconocimiento de caracteres. Nosotros nos hemos centrado en los ejercicios 1 y 2. Se ha seguido la siguiente arquitectura modular jerárquica, con un main que entrena y evalúa y varios módulos independientes para preprocesado como se muestra a continuación:

La organización del proyecto sigue una estructura modular, que facilita la extensibilidad y la separación de responsabilidades:

- **main_prac2_train_ocr.py**: script principal que ejecuta el entrenamiento, carga de datos y evaluación.
- **src/**: contiene todo el código fuente del proyecto.
 - **classifiers/**: implementaciones de los clasificadores.
 - `base.py`: clase abstracta `OCRClassifier`.
 - `svm.py`: clasificador basado en SVM.
 - `rf.py`: clasificador Random Forest.
 - `knn.py`: clasificador K-Nearest Neighbors.
 - `preprocessor.py`: clase `CharImgPreprocessor` para preprocesamiento de imágenes.
 - `eval_utils.py`: funciones para métricas y visualización de resultados (ROC, PR, matriz de confusión).
 - `ocr_training_data_loader.py`: clase para cargar, preprocesar y aplanar las imágenes de entrenamiento por clase.
 - `ocr_test_data_loader.py`: clase para cargar, preprocesar y aplanar las imágenes de prueba por clase.
 - `utils.py`: funciones auxiliares como `flatten_images_dict`, `load_image`, etc.
- **saved_models/**: carpeta donde se almacenan los clasificadores entrenados (`.pkl`).
- **data/**: contiene los conjuntos de entrenamiento y test:
 - `train_ocr/`: imágenes de caracteres individuales para entrenamiento.
 - `test_ocr_char/`: imágenes individuales de test.
 - `test_ocr_words_plain/`: imágenes completas con líneas de texto (no usadas en esta práctica).

2. Ejercicio 1: Entrenamiento y validación del clasificador de caracteres

2.1. Descripción del conjunto de datos

Se han utilizado los siguientes conjuntos:

- **train_ocr**: contiene imágenes con caracteres individuales (dígitos, letras en mayúsculas y minúsculas), con ligeras transformaciones y ruido.
- **test_ocr_char**: conjunto de validación con caracteres similares para evaluar el modelo.

2.2. Carga y preprocesamiento de imágenes

Para el preprocesamiento de las imágenes se ha creado la clase `CharImgPreprocessor`, ubicada en `CharImgPreprocessor`. Esta clase aplica:

- Umbralizado adaptativa usando `cv2.adaptiveThreshold`.
- Detección de contornos y recorte con `cv2.findContours` y `cv2.boundingRect`.
- Redimensionado a 30x30 píxeles.

Para cargar y preprocesar los datos se hace uso de las clases `OCRTrainingDataLoader` (proporcionada en los materiales de la práctica) y `Preprocessor`, respectivamente. En cuanto al preprocesamiento, el código del método `process_image` de la clase `Preprocessor` es el siguiente:

```
def process_image(self, img_gray):
    thresh = cv2.adaptiveThreshold(
        img_gray, 255,
        cv2.ADAPTIVE_THRESH_MEAN_C,
        cv2.THRESH_BINARY_INV, 11, 2
    )

    contours, _ = cv2.findContours(
        thresh, cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )
```

```

bboxes = []
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    if w > 5 and h > 5:
        bboxes.append((x, y, w, h))

if bboxes:
    return bboxes[0]
else:

```

A continuación, realizamos el aplanado para obtener el vector de características en el código principal.

2.3. Entrenamiento del clasificador

Para mantener modularidad y escalabilidad, se implementó la clase base `OCRCClassifier` con métodos abstractos `fit` y `predict` (con la idea de ampliar posteriormente para el ejercicio 2).

```

class OCRCClassifier:
    def fit(self, X, y):
        raise NotImplementedError("Debes implementar el método fit")

    def predict(self, X):
        raise NotImplementedError("Debes implementar el método predict")

    def save(self, path):
        with open(path, 'wb') as f:
            pickle.dump(self, f)

    def load(self, path):
        with open(path, 'rb') as f:
            classifier = pickle.load(f)
        return classifier

    def evaluate(self, X, y):
        y_pred = self.predict(X)
        accuracy = accuracy_score(y, y_pred)
        f1 = f1_score(y, y_pred, average='weighted')

```

- **SVMClassifier**: clasificador SVM con kernel lineal.

2.4. Evaluación del modelo

Para este apartado, tenemos el módulo `eval_utils` con las funciones: `print_metrics` para imprimir Accuracy y los distintos F1-score ; `plot_roc_curve` para la curva ROC y `plot_precision_recall` para la gráfica Precisión-Recall y `plot_confusion_matrix`, aunque esta no la hemos utilizado por ser demasiadas clases.

2.4.1. Métricas de evaluación

Métrica	Valor
Accuracy	0.8809
F1-score macro (todas)	0.5256
F1-score macro (presentes)	0.9052
F1-score ponderado (weighted)	0.9053

Tabla 1: Métricas de rendimiento del modelo

2.4.2. Curva ROC del modelo SVM

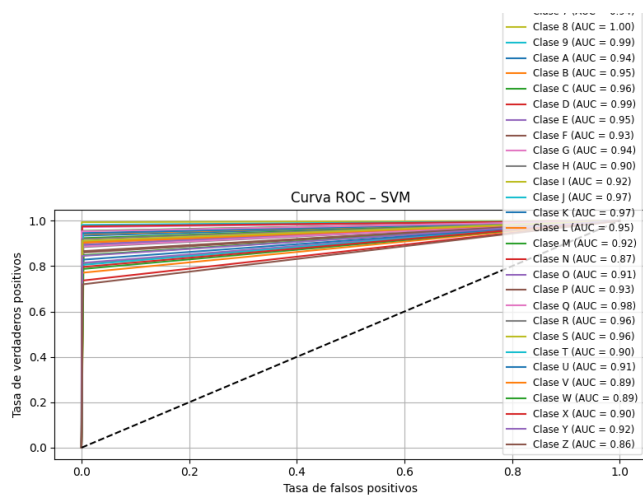


Figura 1: Curva ROC y valor de AUC del modelo SVM

2.4.3. Curva Precision-Recall del modelo SVM

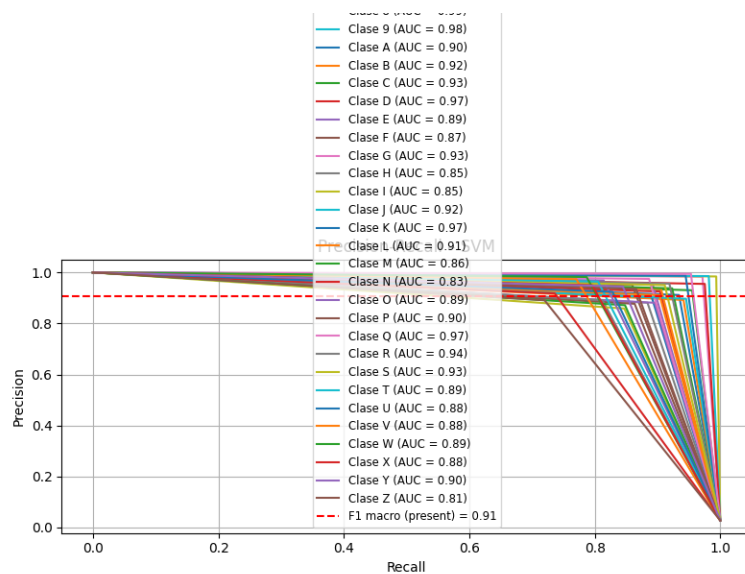


Figura 2: Curva Precision-Recall del modelo SVM

3. Ejercicio 2: Pruebas con alternativas

3.1. Nuevos modelos

Utilizando la misma clase base `OCRCClassifier` hemos creado nuevos clasificadores:

- **RFClassifier**: clasificador Random Forest con 100 árboles.
- **KNNClassifier**: clasificador K-NearestNeighbors con $k=3$.

3.1.1. Métricas comparativas de modelos

Modelo	Accuracy	F1 macro (todas)	F1 macro (presentes)	F1 weighted
KNN	0.9581	0.5628	0.9693	0.9693
RF	0.9616	0.5653	0.9736	0.9736

Tabla 2: Comparación de métricas entre los modelos KNN y Random Forest

3.1.2. Curva ROC de los modelos

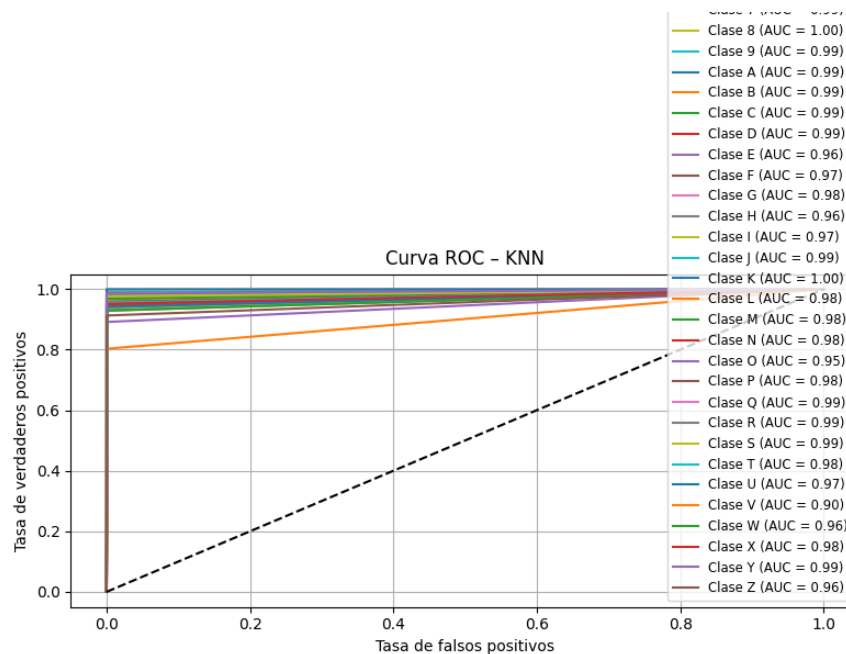


Figura 3: Curva ROC del modelo KNN

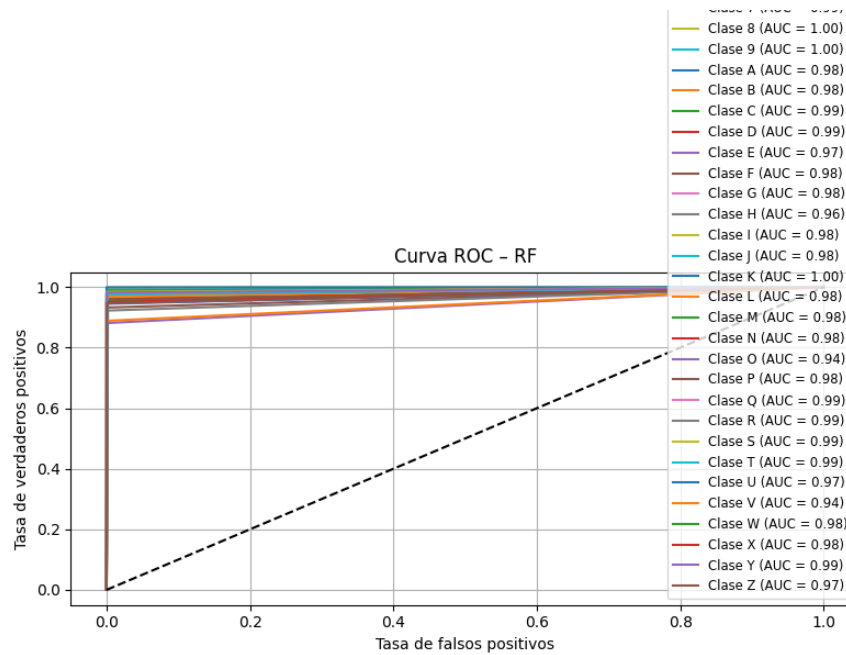


Figura 4: Curva ROC del modelo Random Forest

3.1.3. Curva Precision-Recall de los modelos

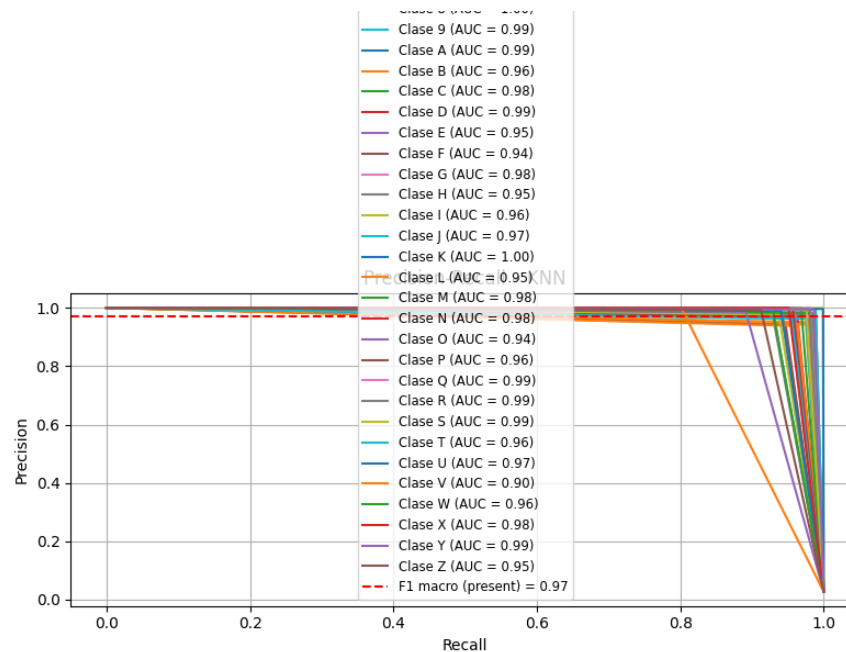


Figura 5: Curva Precision-Recall del modelo KNN

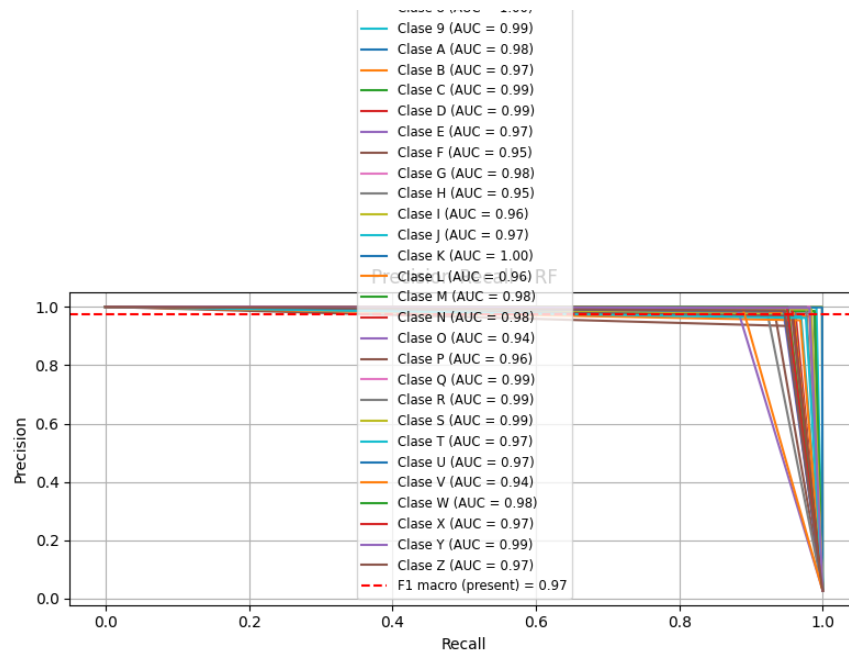


Figura 6: Curva Precision-Recall del modelo Random Forest

3.2. Comparativa de modelos: SVM, KNN y Random Forest

Los tres modelos de clasificación evaluados han sido SVM, KNN y Random Forest.

3.3. Análisis de métricas de rendimiento

Modelo	Accuracy	F1 macro (todas)	F1 macro (presentes)	F1 weighted
SVM	0.8809	0.5256	0.9052	0.9053
KNN	0.9581	0.5628	0.9693	0.9693
Random Forest	0.9616	0.5653	0.9736	0.9736

Tabla 3: Comparación de métricas entre los modelos SVM, KNN y Random Forest

Una vez consultadas las métricas de rendimiento (ver Tabla 3), procedemos a su análisis:

El valor relativamente bajo del *F1-score macro* en los tres modelos se explica porque el conjunto de prueba no contiene letras minúsculas. Este tipo de promedio penaliza las clases ausentes asignándoles un valor de F1 igual a cero. Por esta razón, resulta más representativo considerar el *F1 macro* calculado únicamente sobre las clases presentes, donde todos los modelos muestran un rendimiento elevado (superior a 0.90).

3.3.1. Análisis de las curvas ROC y Precision-Recall

Las curvas ROC y Precision-Recall muestran valores altos para casi todas las clases en los modelos KNN y Random Forest (ver Figuras 3, 4, 5 y 6), mientras que el modelo SVM presenta un rendimiento algo inferior para algunas clases (ver Figura 2).

3.3.2. Tiempo de entrenamiento

En cuanto al tiempo de entrenamiento, cabe destacar que:

- **SVM** requiere un tiempo considerablemente mayor para entrenarse, especialmente con conjuntos de datos amplios y muchas clases. Aproximadamente un 50% del total..
- **KNN** tiene un entrenamiento prácticamente inmediato (cerca del 15 % del tiempo total de entrenamiento), aunque sus predicciones son más lentas.
- **Random Forest** presenta un equilibrio adecuado, con un tiempo de entrenamiento moderado (alrededor del 35 %) y resultados muy competitivos.

3.3.3. Conclusión

De los tres modelos evaluados, **el modelo Random Forest es el más recomendable** en este contexto. No solo ofrece la mejor combinación de métricas cuantitativas, sino que también muestra un comportamiento muy estable en las gráficas ROC y Precision-Recall. Además, mantiene tiempos de entrenamiento razonables frente al elevado coste computacional del modelo SVM.